Reliable Candidate Selection for Budget-Constrained Machine Learning Agents

DATS-SHU-200 Data Mining
Homework 3: Research Proposal

1 Introduction

Large language model (LLM) agents can automate parts of machine learning engineering, such as writing training code, debugging failures, and refining existing solutions. A central difficulty is deciding which proposed solution merits an expensive execution. An agent may generate several modifications to a training pipeline, yet only a few can be trained and evaluated within the available time. Executing every proposal consumes resources, whereas discarding proposals prematurely can remove the best solution. This project studies how an agent should allocate a limited budget among comparing, testing, and improving candidate programs.

Existing agent systems offer two complementary approaches to this problem. AIDE formulates machine learning engineering as tree search over code solutions, in which the agent drafts, debugs, and improves programs using execution feedback (Jiang et al., 2025). Observed performance thus informs subsequent decisions, but only after candidate programs are run. ForeAgent instead introduces a *Predict-then-Verify* loop (Zheng et al., 2026). During improvement, it generates multiple candidates and asks an LLM to compare their code, given the task description and a data analysis report. A confidence-gated selection procedure then filters the candidates before execution. This reduces the need to run every proposal while retaining execution feedback for the candidates that are actually verified.

Judgments can conflict, and a single confident mistake can eliminate a strong program. Which evidence helps an agent decide what to train next? After reproducing ForeAgent, we tested whether the recorded scores of similar programs helped it choose among candidates in the authors' corpus. We also replayed agent logs to see when validation scores already available during a run changed that choice. The corpus provides outcomes for both alternatives; a live run records outcomes only for the programs actually executed.

We first reproduced ForeAgent's released candidate-selection component. Its released agent trajectories, which contain historical candidate batches and observations, allow a direct check of the original control logic. Section 3 reports three complementary experiments: conditional historical replay, offline selection-quality evaluation, and a single-task end-to-end run. The public Preference Corpus, containing 18,438 candidate pairs across 26 tasks, supplies saved judgments and recorded execution scores for the offline evaluation (Zheng et al., 2026). MLE-bench provides the broader machine learning engineering task setting (Chan et al., 2024). Historical-output agreement, prediction accuracy, and end-to-end task performance each provide a different level of evidence.

2 Baseline Explanation

2.1 Problem formulation and notation

Let I denote a task description, D its dataset, and C a candidate program implementing a machine learning workflow, and let $M(C; I, D)$ denote the task-specific evaluation metric obtained after executing C. We define an oriented utility $u(C)$ for which higher values are always better: $u(C) = M(C; I, D)$ for a maximized metric such as AUC, and $u(C) = -M(C; I, D)$ for a minimized

metric such as log loss. The underlying engineering objective is to find

$$C^* \in \arg\max_{C \in \mathcal{C}} u(C), \tag{1}$$

where $\mathcal{C}$ is the space of possible programs. Because obtaining $u(C)$ requires execution, ForeAgent predicts relative quality before incurring this cost. The notation describes the optimization objective only; the true utility is unavailable to the predictor at decision time.

For candidates C_i and C_j, the comparison input consists of I, a data analysis report D_{rep}, the two programs, and a prompt P. A pretrained LLM with fixed parameters θ returns

$$(z_{ij}, \widehat{y}_{ij}, c_{ij}) = f_\theta(I, D_{\text{rep}}, C_i, C_j; P), \tag{2}$$

where z_{ij} is the generated reasoning, $\widehat{y}_{ij} \in \{0, 1\}$ identifies the preferred input position, and $c_{ij} \in [0, 1]$ is the reported confidence. The evaluation label indicates which candidate achieved the better recorded score. Because the index refers to a position within the presented pair, it must be mapped back to candidate identity when comparisons are reused. Moreover, c_{ij} applies only to this particular judgment; confidence values from unrelated pairs cannot be treated as a global ranking of candidates.

2.2 System architecture: prediction followed by verification

ForeAgent builds on AIDE's search over program nodes, in which executed nodes provide code, runtime observations, and measured performance for later drafting, debugging, or improvement. ForeAgent modifies the *improvement* stage by inserting prediction-based filtering between generation and execution (Zheng et al., 2026, Section 6.2):

$$\begin{aligned}\text{Executed parent} &\longrightarrow \text{Candidate generation} \longrightarrow \text{LLM comparisons} \\ &\longrightarrow \text{Confidence-gated selection} \longrightarrow \text{Execution and feedback.}\end{aligned}$$

The published setup proposes $m = 10$ improvements to a parent solution, applies a confidence threshold $\tau = 0.7$, and selects the top rank group with $k = 1$; the selected candidates are then subject to the execution policy described below. Execution here means training and evaluating a candidate program, as distinct from calling the LLM to compare code. Successful verification returns measured feedback to the search, so the full agent retains interaction with its computational environment.

The predictor is grounded in task and dataset context rather than code alone. The paper constructs its data reports through a profiling, verification, and verbalization pipeline: generated analysis code is executed, the resulting logs are checked, and the observations are converted into a textual report (Zheng et al., 2026, Section 3.4). The report helps the LLM assess whether a pipeline suits the data; it is part of the prediction input, not the candidate's hidden evaluation score. The offline baseline reuses the released reports rather than regenerating them.

2.3 Inference-time selection and execution rules

The core method relies on pretrained LLM inference; it neither fits an additional reward model nor updates model parameters. The relevant technical specification is therefore its decision rule rather than a new training loss. The confidence gate accepts a comparison when

$$a_{ij} = \mathbf{1}\{c_{ij} \geq \tau\}, \qquad \tau = 0.7. \tag{3}$$

A high-confidence winner can advance, whereas an inconclusive judgment does not immediately establish a loser. The gate acts on self-reported confidence; it is not a calibrated probability model fitted by this project.

Tournament aggregation. In the fixed released implementation, each round compares adjacent active candidates, and the unpaired candidate in an odd-sized round receives a bye. Confident winners advance and losers are eliminated, while inconclusive pairs enter a pending list. Pending candidates are compared with randomly chosen advanced candidates; if none has advanced, one pending candidate advances directly. A pending winner advances without removing its already-advanced opponent, and a further inconclusive outcome also advances the pending candidate. The process stops when one survivor remains or the round count exceeds $2m$. The final ordering places survivors first, followed by wins and later rounds; consecutive candidates with equal $(\text{wins}, \text{round})$ share a displayed rank (ForeAgent authors, 2026a).

Selection and probabilistic execution. Let r_i be the zero-based displayed rank. The multi-candidate branch marks candidates with $r_i < k$ as eligible and applies an independent execution probability $p_{\text{exec}} = 0.2$ to each. If these tests produce an empty execution set, the original code forces execution of the first rank-0 candidate in sorted order (ForeAgent authors, 2026a). Formally, let $B_i \sim \text{Bernoulli}(p_{\text{exec}})$ and let i_0 index that first rank-0 candidate. The preliminary and final execution indicators are

$$q_i = \mathbf{1}\{r_i < k\}\, B_i, \qquad e_i = q_i + \mathbf{1}\left\{\sum_j q_j = 0\right\} \mathbf{1}\{i = i_0\}. \tag{4}$$

The original multi-candidate policy thus executes at least one candidate, and ties may permit several executions. Together, the probability gate and the mandatory fallback define the execution decision used in Section 3.

2.4 Baseline evaluation and the scope of this project

The paper evaluates two distinct capabilities. First, its *Data-centric Solution Preference* task measures how well an LLM predicts pairwise winners. The Preference Corpus contains 895 curated solutions and 18,438 pairs from 26 tasks spanning computer vision, natural language processing, and data science. The principal metric is micro-averaged pairwise accuracy,

$$\text{Acc}_{\text{pair}} = \frac{1}{N} \sum_{\ell=1}^{N} \mathbf{1}\{\widehat{y}_\ell = y_\ell\}, \tag{5}$$

where N is the number of evaluated pairs and y_ℓ is the recorded better-candidate label. The authors report 61.5% accuracy for DeepSeek-V3.2-Thinking, averaged over three full-corpus runs (Zheng et al., 2026, Table 2). On the same 246-pair subset across 26 tasks, our run correctly predicted 136 winners (55.28%) and one released author run predicted 130 (52.85%).

Second, the full ForeAgent system is compared with AIDE on five MLE-bench tasks under a 12-hour limit, with three runs per task. This evaluation measures final task performance by Beat Ratio, the proportion of human leaderboard contestants outperformed, alongside analyses of execution time and search breadth (Zheng et al., 2026, Sections 6.3–6.4). Pairwise accuracy does not establish these end-to-end outcomes: a predictor can make locally correct comparisons yet select poorly within a candidate pool, and good offline selection does not establish actual savings in training time.

For the original controller, we replayed archived decisions, evaluated selected programs against their recorded corpus scores, and ran the complete agent for 12 hours on Stanford COVID Vaccine. We then built C0, a corrected version of the tournament that maps cached comparisons back to candidate identities and binds the selected submission to its saved prediction file. C0 provided the control for subsequent experiments on how to choose a program from a fixed pool.

3 Baseline Reproduction

3.1 Objectives and experiment scope

Three experiments address complementary questions: whether the released controller reproduces archived decisions, how well it selects candidates from fixed pools, and what final task performance it achieves in a live run. All three use the original implementation at commit `c4d52cf` and retain its cache behavior, confidence gate, execution probability, and mandatory execution fallback. The historical experiment is described first; the offline and online results follow in Sections 3.6 and 3.7.

The historical reproduction tests whether ForeAgent's released ranking and execution-control logic reproduces its recorded outputs when given the original candidate batches, generation order, and saved observations. The original cache implementation is preserved (ForeAgent authors, 2026a). The comparison backend supplies recorded responses by batch position, and the execution branch receives the probability outcomes recorded in the logs. The experiment is therefore a *conditional historical-trace replay*: it verifies the state updates produced from those observations.

We froze the historical replay on November 7, 2026. The release manifest records SHA-256 checksums for the source snapshots, inputs, scripts, and verification outputs used to check agreement.

The original controller also produced the fixed-pool results in Section 3.6 and the Stanford submission in Section 3.7.

3.2 Released records and experimental setup

The inputs come from the authors' public `agent_runs/ForeAgent/` release, fixed at dataset revision `6b322cb` (ForeAgent authors, 2026b). We inspected all 15 released ForeAgent runs, three for each of five MLE-bench tasks. For each run, the inputs comprise its actual `config.yaml`, full `journal.json`, and full `aide.log`; these 45 original text files were checked against their recorded sizes and hashes. Table 1 summarizes recovery coverage.

Table 1: Coverage of the released historical runs. Logged batches include those with interrupted generation or final-journal gaps. Each recovered batch contains ten candidates.

Task	Runs	Logged batches	Recovered	Candidates
Aerial Cactus Identification	3	14	12	120
Histopathologic Cancer Detection	3	5	4	40
Stanford COVID Vaccine	3	45	42	420
Statoil Iceberg Classification	3	60	57	570
Ventilator Pressure Prediction	3	16	14	140
Total	15	140	129	1,290

The saved configurations agree on the relevant settings: ten improvement candidates, confidence threshold 0.7, Top-$k = 1$, execution probability 0.2, and a 12-hour agent time limit. The comparison model is labeled DeepSeek-V3.2-Thinking, with temperature 1.0. These model settings describe the source runs; the replay consumes their saved responses. Local callbacks record execution decisions, while training and inference observations come from the archived runs. The replay makes zero additional model API calls and zero candidate-training runs.

Batch eligibility is determined by record availability. Of 140 logged batches, 129 have completed comparison histories and corresponding final-journal nodes. The remaining 11 are retained in a separate coverage ledger: six end during generation, and five contain generated nodes absent from the final journal. This rule depends on structural completeness alone and was applied independently of candidate scores or agreement results.

3.3 Recovery and replay procedure

Candidate identity, grouping, and order. Log messages of the form `Generated improvement candidate i/10` identify the original generation order. Candidate IDs are resolved to their full journal entries and actual source code. The `node2parent` map verifies a shared parent within each recovered batch, and creation-time ordering provides a second check on the logged sequence. Because the journal's stored list reflects post-selection saving and re-appending, the replay input order is established from generation logs and timestamps.

Historical comparisons. Each candidate in a batch stores the same `wm_comparison_results` sequence, which records winner position, confidence, round, and validity. A local provider passes these archived results to the original `tournament_rank()` calls in their recorded order. The replay checks response counts, round indices, and agreement between confidence and the 0.7 validity gate. It also checks 8,725 ordinary comparisons against the logged participant IDs and outcomes, and 15,148 revival comparisons against the logged pending-candidate identities and outcomes.

The records used for this replay did not identify the original controller's random seed or RNG state. In revival matches, the advanced opponent is represented by an explicitly marked placeholder choice generated with demonstration seed 0, and responses are supplied by recorded batch position. The checked historical information identifies the pending candidate and the outcome but provides only partial evidence about the advanced opponent. The reported agreement therefore covers the control flow conditional on the recorded responses.

Execution decisions and mandatory fallback. The original execution statements are extracted directly from `agent.py`, lines 700–767, and checked for AST equality. Logged probability-trigger

outcomes drive the execution tests of eligible candidates. The original code then applies its mandatory fallback: whenever this stage yields an empty execution set, the first rank-0 candidate in sorted order is executed. Local callbacks record the selected identities. The resulting execution set is checked against the journal's `is_skipped` flags, and its order against execution-parsing log messages. Because observed probability outcomes are supplied, this check is conditional on the historical draws; what it verifies is Top-k handling, candidate selection, and fallback processing.

3.4 Results and agreement with the released traces

For all 129 recovered batches, the original implementation reproduces the recorded candidate ranks, round metadata, sorted order, and final execution decisions; Table 2 states the comparison units explicitly. Across these batches, the replay consumes 23,873 comparison records, including repeated rounds and cached observations. These records are observations from the historical trajectory, and repeated comparisons contribute to the recorded control flow.

Table 2: Agreement conditional on archived comparison responses and logged probability outcomes. Candidate-level and batch-level denominators count different units.

Checked output	Agreement	Rate
Candidate displayed rank	1,290 / 1,290	100%
Candidate round metadata	1,290 / 1,290	100%
Complete sorted candidate order	129 / 129 batches	100%
Final execution set	129 / 129 batches	100%
Final execution order	129 / 129 batches	100%

The mandatory execution fallback occurs in 87 of the recovered batches, matching the logged events. For example, in the first recovered batch of the Aerial Cactus run with UUID prefix `c5d8f2ae`, candidate `69daaf39` receives rank 0. Its probability test is recorded as skipped, after which the original fallback selects it. Both the resulting execution identity and the final journal flag agree with the replay.

The 100% values denote *historical-output agreement*. Recorded winner positions are supplied as observations, and true candidate-performance scores play no role in recovering ranks. The result therefore establishes faithful processing of these observed histories. Pairwise prediction accuracy, selection quality against executed scores, and end-to-end agent efficiency are separate evaluation questions.

3.5 Interpretation and reproducibility artifacts

The replay establishes an implementation baseline for the candidate-selection component: under the released control rules, the recovered historical observations lead to the same ranking and execution outputs. The paper's full-corpus prediction accuracy and five-task online results provide broader reference experiments, whereas this section contributes a direct consistency check against the released agent trajectories (Zheng et al., 2026). The archived responses already reflect the source runs' inference and cache behavior; fresh model judgments and reconstruction of the original revival opponents require their own experimental conditions.

The accompanying `baseline_freeze.json` pins the source, data, scripts, and verified outputs by SHA-256. The archive contains the full logs and journals, the coverage ledger, and per-batch comparisons, and its `replay_author.py` and `verify_and_report.py` scripts reproduce and check these results.

3.6 Offline selection quality: `corpus26-v2`

Setup. This experiment asks whether the original tournament selects candidates with better recorded execution scores than uniform random choice. It uses candidates and one set of published pairwise judgments from the Preference Corpus. Each task contributes 100 candidate-pool instances, each replayed three times with fixed orders and seeds, giving 2,600 pool instances, 2,164 distinct candidate sets, and 7,800 original-controller replays across 26 tasks. Twenty-one tasks use pools of ten

candidates; the other five use pools of 3, 3, 5, 6, and 8. Pool construction uses finite-score availability and pairwise-feedback coverage; score magnitude and candidate rank are reserved for evaluation after selection.

The settings are confidence threshold 0.7, Top-$k = 1$, and execution probability 0.2. The original cache is retained and cleared before each replay, and comparisons run serially for deterministic replay. Saved judgments replace new LLM responses, and candidate filenames serve as identity stubs in the cache. Execution is represented by recording the selected identities and looking up their existing scores. Both the probability stage and the original mandatory fallback are included. The experiment makes zero additional API calls and zero candidate-training runs.

Metrics. Let S be the displayed rank-0 group, $B = \arg\max_{i \in P} u_i$ the best candidates in pool P, and E the final simulated execution set. The selection metrics average over a uniform single choice within S:

$$H = \frac{|S \cap B|}{|S|}, \qquad R = \frac{\max_{i \in P} u_i - |S|^{-1} \sum_{i \in S} u_i}{\max_{i \in P} u_i - \min_{i \in P} u_i}. \tag{6}$$

Here R is the normalized regret for a pool with a nonzero utility range; lower is better. The random reference chooses uniformly from the same pool, with hit probability $|B|/|P|$. We average repeats within each pool, pools within each task, and then the 26 tasks with equal weight. The execution diagnostic is $\mathbf{1}\{E \cap B \neq \varnothing\}$ under the complete original policy, which separates single-choice selection quality from a policy that may execute several tied candidates.

Table 3: Offline selection quality of the original ForeAgent on the fixed corpus pools. Values are task-macro averages; uniform random choice uses the same candidate pools.

Metric	Original ForeAgent	Uniform random choice
Uniform-tie best-hit (H, higher better)	20.43%	12.53%
Normalized regret (R, lower better)	0.2810	0.3674

Results and interpretation. Table 3 shows a descriptive advantage of 7.89 percentage points in best-hit probability over random choice, together with lower regret. The complete execution simulation includes a best candidate in 21.60% of replays and executes 1.028 candidates per replay on average. Every replay has a final execution, and the mandatory fallback is used in 74.05% of replays. These values describe this fixed-feedback offline experiment.

The 7,800 replays reuse candidates and judgments. Each pool combines programs from the corpus, while a live batch contains improvements to one parent. Table 3 measures the original controller on those fixed pools. The protocol and verification records are in `experiments/corpus26-v2/`.

Subsequent corpus diagnostic (November 10). We evaluated C0 on a separate set of 425 pools from the 26 development tasks. With DeepSeek judgments, its task-mean normalized regret was 0.2642. When the number of reference scores was matched, adding scores from similar programs in the same task lowered regret to 0.2554 if drafts from the same original runs were eligible. Restricting references to other runs gave 0.2654. In 287 pools, both methods had at least three reference scores; in the remaining pools, both used C0. These are offline decisions scored with recorded outcomes. The Stanford run below used the original agent.

3.7 Single-task end-to-end reproduction

Task and environment. Run `stanford-autodl-v2` executes the original ForeAgent on Stanford COVID Vaccine, a task that predicts RNA degradation-related measurements. The prepared data contain 2,160 training sequences and 240 test sequences; the Kaggle archive and four prepared files match the MD5 checksums recorded in the fixed release. The released task data report is reused. The runtime uses the pinned author OCI image through PRoot on an AutoDL RTX 4090, while the controller and model requests run on a Mac. DeepSeek-V3.2 generates code, DeepSeek-V3.2-Thinking compares candidates, and `gpt-4o-mini-2024-07-18` parses execution feedback. The DeepSeek requests use the SiliconFlow service with model-name and thinking-mode mapping; the feedback requests use the official OpenAI endpoint.

The run retains ten improvement candidates, confidence threshold 0.70, Top-$k = 1$, execution probability 0.20, and the original cache and fallback. The 43,200-second wall-clock budget ran from November 8, 2026, 01:22:02 to 13:22:02, China Standard Time, and covers generation, API waits and retries, selection, execution, and feedback. The original algorithm selects its final submission by its internal validation metric; external evaluation uses the official private-answer scorer after the run.

Run outputs. The completed journal contains 82 nodes: 22 executed and 60 skipped. There are 19 completed outer iterations and seven complete tournament batches; because an outer iteration may generate several nodes, these counts use different units. The retained node is `84c79a8d`, with a parsed internal validation value of 0.4378, and its submission contains 25,680 valid prediction IDs. The official private-set error is **0.26640**; an independent, ID-aligned calculation gives 0.2663989312. The scored subset contains 16,320 sequence-position rows. The metric is the mean columnwise root mean squared error (MCRMSE) across reactivity, deg_Mg_pH10, and deg_Mg_50C; lower is better. The internal validation value and the private-set score are separate evaluations.

Table 4: Same-task endpoint comparison. The authors' three released final submissions and this run are scored against the same private answers. Each entry is a single run; lower MCRMSE is better.

Stanford COVID Vaccine submission	Private-set MCRMSE
Author run `01abb886`	0.25768
Author run `5d12c313`	0.24320
Author run `cbb2a0b0`	0.27497
This reproduction (one 12-hour run)	**0.26640**

Comparison and operating conditions. The Stanford score falls within the range of the authors' three runs, which averaged 0.25862 (Table 4). The paper covers five tasks with three runs per task. Our single run establishes that the original loop produced a valid submission with a comparable score on Stanford. Measuring run-to-run variation and the contribution of filtering requires repeated runs and a matched execution baseline.

Hardware, model services, and network latency differed between runs. The experimenter reported that Mac screen sleep and a VPN disconnection disrupted model API access. One feedback request took 13,491.88 seconds (about 3 hours 45 minutes), including processing, retries, and interruption-related waiting. The entire interval counted toward the standard 12-hour budget. The logs record the request's duration but give no separate measure of the VPN outage. A subsequent extension ended at the user's request with zero new completed nodes and the same prediction hash. Table 4 reports the standard 12-hour result.

Scoring and artifact checks. The original scorer and an independent ID-aligned MCRMSE calculation agree for this submission and all three author submissions. Shuffling prediction rows changes the original pandas-based score, whereas the ID-aligned calculation is invariant. Verification covered unique IDs, required columns, finite predictions, and agreement between the two methods. The verification records document the original scorer's row-order requirement.

We verified 484 Git files against the pinned commit and linked the retained journal node to its final prediction file by SHA-256. Run counts, timings, submission hashes, and author references are archived in `experiments/online-stanford-v2/`. Independent source, data, artifact, and scoring checks are in `experiments/environment-acceptance/`.

4 Future Plan

Research objective. Within a fixed time and compute budget, which observations help an agent choose what to run or improve next? We will evaluate immediate choices against measured candidate performance and evaluate full runs by their final submission scores and costs. C0, our corrected version of the original tournament, provides the control for method changes. The Stanford reproduction in Section 3.7 used the original agent.

First experiment. Use past executions to choose the next program. For each candidate, retrieve the three most similar programs among up to ten earlier executions from the same run, using TF–IDF cosine similarity over code tokens and adjacent token pairs. Weight the neighbours' validation ranks by their similarity. Combine the resulting candidate rank with its judge rank, giving each equal weight. The judge rule prioritizes the C0 top group, then accepted win fraction and number of accepted opponents. With fewer than three usable earlier scores, both methods use C0.

In a pilot on 26 development tasks, programs from outside the current pool and their corpus scores stood in for a live agent's execution history. Equal-weight fusion, a prespecified secondary method, lowered task-mean normalized regret from 0.2642 to 0.1966 with DeepSeek judgments and from 0.2725 to 0.2030 with pooled GPT judgments. Breaking only C0 ties, the primary method, gained 0.0093 on GPT against a 0.01 advancement criterion. The next test uses validation scores actually available at selection time.

CPU screen on real search histories. We checked 136 batches of ten siblings from 16 runs across five tasks. At least three earlier validation scores were available in 130 batches and ten were available in 60. Only six batches offered enough history and two or more executed siblings with distinct comparable outcomes. On their 13 pairs, memory alone ordered 29.6% correctly under task-balanced averaging, versus 50.8% after shuffling history scores. The original tournament tied on every observed pair.

We replayed C0 with the archived parsed judgments and a cap of 13 distinct feedback reads. Its top group could be reconstructed in **128 of 136 batches**. Using the same fixed tie order for both selectors in each batch, fusion changed the chosen program in **80 of 128 (62.5%)**. In 47 of these batches the top groups were disjoint, so every tie order changes the choice. Six other batches contained duplicate programs outside the C0 replay interface. The two remaining batches required unobserved judgments. In one Aerial Cactus batch all admissible completions implied a switch, even though the baseline top group was uncertain. Thus among the 130 eligible batches we established **81 switches, 48 agreements and one unresolved choice**. Among the 80 observed choice changes, zero pairs had usable historical validation scores for both selected programs. Paired execution supplies the missing outcomes.

Measure the value of a changed choice. Freeze the selectors, task sample, scorer, seeds and training limits before collecting outcomes. Give C0 and fusion the same candidates and comparisons and record the program each would run. Each agreement contributes zero score difference. At disagreements train both choices on the same split and budget. Report the disagreement rate over all batches, the paired score change within each task, failures and elapsed time. Repeat a subset with another seed to estimate training variability. A control that shuffles historical scores tests how much the association between a program and its score contributes. Charge runs made solely to evaluate a counterfactual choice to the evaluation budget. If the paired gains recur across tasks and exceed training variation at comparable cost, freeze the method for full agent runs on fresh tasks.

Other directions. Three further directions address different decisions. If retrieval scores prove weak, supply the pairwise judge with scored examples and measure how well it predicts execution outcomes relative to the original prompt. At uncertain decisions, compare spending the remaining budget on another code comparison with running a short training test. If the best candidate in a batch already performs poorly, vary which parent is expanded or how changes to its code are proposed, keeping the number of generated programs and training budget fixed. Each test measures the quality of the selected program and the cost of obtaining it.

References

Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-Driven Exploration in the Space of Code. *arXiv:2502.13138*, 2025. https://arxiv.org/abs/2502.13138.

Jingsheng Zheng, Jintian Zhang, Yujie Luo, Yuren Mao, Yunjun Gao, Lun Du, Huajun Chen, and Ningyu Zhang. Can We Predict Before Executing Machine Learning Agents? In *Proceedings of ACL 2026 (Volume 1: Long Papers)*, pp. 3941–3974, 2026. https://aclanthology.org/2026.acl-long.182/.

Jun Shern Chan et al. MLE-bench: Evaluating Machine Learning Agents on Machine Learning Engineering. *arXiv:2410.07095*, 2024. https://arxiv.org/abs/2410.07095.

ForeAgent authors. Released implementation of Predict Before Execute. GitHub repository, fixed commit c4d52cf (full identifier in the URL below). Relevant files: world_model.py, agent.py, and config.yaml under mle-bench/agents/aide/. https://github.com/zjunlp/predict-before-execute/tree/c4d52cf99bd870d830b456ac7c0684aec1aef375.

ForeAgent authors. PredictBeforeExecute: released ForeAgent execution records. Hugging Face dataset, fixed revision 6b322cb; subdirectory agent_runs/ForeAgent/. https://huggingface.co/datasets/zjunlp/PredictBeforeExecute/tree/6b322cb88bdbcb2b2d3897ec7d0ded94a5bb2d06/agent_runs/ForeAgent.

Generative AI use. OpenAI Codex assisted with source and log inspection, Python replay and verification tooling, English drafting, and LaTeX typesetting. Requests included (translated from Chinese): "Replay the original version using the authors' real candidate groups, order, and saved judgments, and check agreement" and "Freeze this part and add it to HW3's Baseline Reproduction using the previous NeurIPS template." A further request was to incorporate the corpus26-v2 offline selection results and the new end-to-end run while retaining the 129-batch historical replay. Original prompts and affected components are recorded in the accompanying AI_USAGE.txt.

Claude Code assisted with the corpus pilot, and OpenAI Codex discussed the plan with Claude in an existing Claude Desktop chat. Codex checked the pilot summaries, wrote the chronological replay and selection audit, and drafted Section 4. The follow-up request was to keep the plan broad, list execution memory first, and use subagents to check on CPU how many of the 136 batches support a C0 reconstruction and how often adding memory changes the choice (translated from Chinese). Claude Opus 5.5 edited the English prose in Sections 1–3.